

RankGPT Prompt 模板

这是RankGPT，一款智能助手，专门用于根据查询的相关性对段落进行排序。

以下是{{num}}段文字，每段都有一个数字标识符[]。我将根据查询内容对它们进行排序：{{query}}

[1] {{passage_1}}
[2] {{passage_2}}
(更多段落) ...

查询是：{{query}}

我将根据查询对上述{{num}}段文字进行排序。这些段落将按照相关性降序列出，使用标识符表示，最相关的段落将列在最前面，输出格式应该是[] > [] > 等，例如，[1] > [2] > 等。

对{{num}}段文字的排序结果（仅限标识符）是：

图 6.16: RankGPT Prompt 模板图。

重排任务。这种方法可以利用大语言模型优良的深层语义理解能力，从而取得了良好的表现。RankGPT[47] 是基于上下文学习的重排方法中的代表性方法。其使用的 Prompt 模板如图6.16所示。在重排任务中，输入文档长度有时会超过上下文窗口长度的限制。为了解决该问题，RankGPT 采用了滑动窗口技术来优化排序过程。该技术将所有待排序的文档分割成多个连续的小部分，每个部分作为一个窗口。整个排序过程从文档集的末尾开始：首先，对最后一个窗口内的文档进行排序，并将排序后的结果替换原始顺序。然后，窗口按照预设的步长向前移动，重复排序和替换的过程。这个过程将持续进行，直到所有文档都被处理和排序完毕。通过这种分步处理的方法，RankGPT 能够有效地对整个文档集合进行排序，而不受限于单一窗口所能处理的文档数量。

6.4 生成增强

检索器得到相关信息后，将其传递给大语言模型以期增强模型的生成能力。利用这些信息进行生成增强是一个复杂的过程，不同的方式会显著影响 RAG 的性能。本节将从如何优化增强过程这一角度出发，围绕四个方面展开讨论：（1）何时增强，确定何时需要检索增强，以确保非必要不增强；（2）何处增强，确定在模型中

的何处融入检索到的外部知识，以最大化检索的效用；(3) **多次增强**，如何对复杂查询与模糊查询进行多次迭代增强，以提升 RAG 在困难问题上的效果；(4) **降本增效**，如何进行知识压缩与缓存加速，以降低增强过程的计算成本。

6.4.1 何时增强

大语言模型在训练过程中掌握了大量知识，这些知识被称为**内部知识 (Self-Knowledge)**。对于内部知识可以解决的问题，我们可以不对该问题进行增强。不对是否需要增强进行判断而盲目增强，不仅不会改善生成性能，还可能“画蛇添足”引起**生成效率**和**生成质量**上的双下降。对**生成效率**而言，增强文本的引入会增加输入 Token 的数量，增加大语言模型的推理计算成本。另外，检索过程也涉及大量的计算资源。对**生成质量**而言，因为检索到的外部知识有时可能存在噪音，将其输入给大语言模型不仅不会改善大语言模型的生成质量，反而可能会生成错误内容。如图 6.17 所示，对于“树袋熊一般在哪里生活？”这个问题，大语言模型可以直接给出正确答案。但是，当我们为它提供一段与树袋熊名称非常相似的动物袋熊的外部知识，如图 6.18 所示，大语言模型给出了错误答案，因为大语言模型将知识文本中关于袋熊的信息错误地理解为树袋熊的相关信息。综上，判断大语言模型何时需要检索增强，做到非必要不增强，可以有效的降低计算成本并避免错误增强。

判断是否需要增强的核心在于**判断大语言模型是否具有内部知识**。如果我们判断大模型对一个问题具备内部知识，那么我们就可以避免检索增强的过程，不仅降低了计算成本，而且还可以避免错误增强。判断模型是否具有内部知识的方法可以分为两类：(1) 外部观测法，通过 Prompt 直接询问模型是否具备内部知识，或应用统计方法对是否具备内部知识进行估计，这种方法无需感知模型参数；(2) 内部观测法，通过检测模型内部神经元的状态信息来判断模型是否存在内部知识，这种方法需要对模型参数进行侵入式的探测。

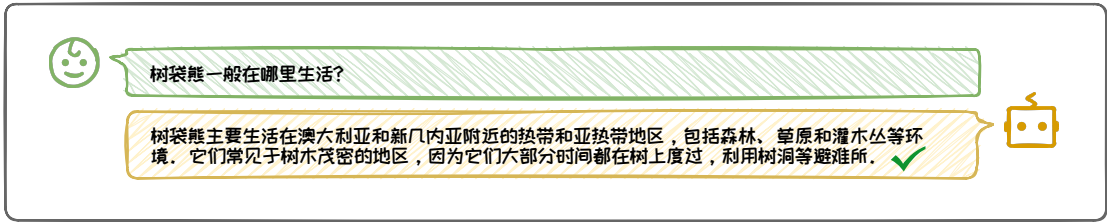


图 6.17: 模型已知问题示例。

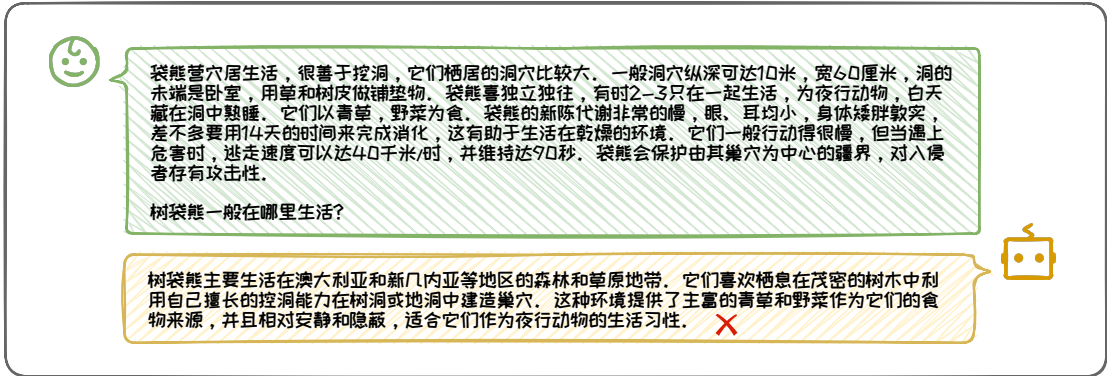


图 6.18: 知识文档损害性能示例。

1. 外部观测法

外部观测法旨在不侵入模型内部参数的情况下，通过直接对大语言模型进行询问或者观测调查其训练数据来推断其是否具备内部知识。这种方法可以类比为人类面试的过程。面试官在评估应聘者的知识和能力时，通常会询问并观察其反应、浏览其过往教育经历等方式，以判断应聘者是否具备足够的专业知识。对于大语言模型，我们可以通过两种问询的方式来判断大语言模型是否具备相应的内部知识：（1）Prompt 直接询问大语言模型是否含有相应的内部知识；（2）反复询问大语言模型同一个问题观察模型多次回答的一致性。此外，我们也可以通过翻看大语言模型的“教育经历”，即训练数据来判断其是否具备内部知识。但是，许多大语言模型的训练数据并未公开，无法直接观测它们的训练数据。在这种情况下，可以通过设计伪训练数据统计量来拟合真实训练数据的分布，从而间接评估模型对特定知识的学习情况。接下来将对这三种方式进行详细介绍。

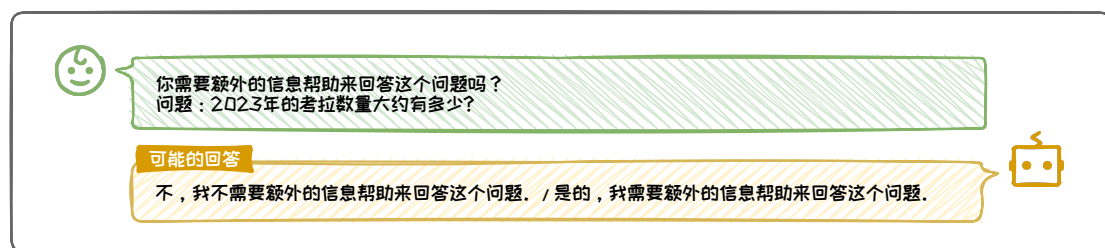


图 6.19: 直接询问的 Prompt 示例。

(1) 询问

我们可以通过直接询问和多次询问两种询问方法来判断大语言模型是否具备内部知识。在直接询问时，我们可以编写 Prompt 直接询问大语言模型是否需要外部知识，如图 6.19 所示。然而，大语言模型很难做到“知之为知之”，其存在“过度自信”的问题：对于无法回答的问题，大语言模型经常认为自己不需要外部知识的帮助，因此这种判断方式的准确率较低。此外，采用多次询问时，我们可以通过让大语言模型重复多次地回答同一个问题，然后根据其回答的一致性判断其是否具备内部知识 [34, 40]。如果模型不具备相应的内部知识，那么每次的输出会较为随机，从而导致多次回答的一致性不强；反之，如果模型具备内部知识，其每次都会回答正确的知识、随机性较弱，输出则会有更高的一致性。然而，这种方式同样面临着模型因过度自信而“执拗”地一直给出相同的错误答案的情况。并且，多次询问还需要耗费大量的时间和计算资源，在实际使用中可行性较低。

(2) 观察训练数据

通过询问来判断内部知识的方法存在的模型回答可靠性较低的问题。我们可以转向观察更为可靠的**训练数据**。大语言模型所具备的内部知识来源于其训练数据。因此，如果模型的训练数据中不包含当前问题的相关信息，那么，模型自然也就未曾习得相应的知识。此外，类比人类的学习模式，我们通常对常见或反复学习的知识掌握程度更好，而对低频的知识则较弱。因此，对于训练数据中出现频率很低的知识，模型对它们的学习程度可能也是比较低的，即**知识在训练数据中的出现**

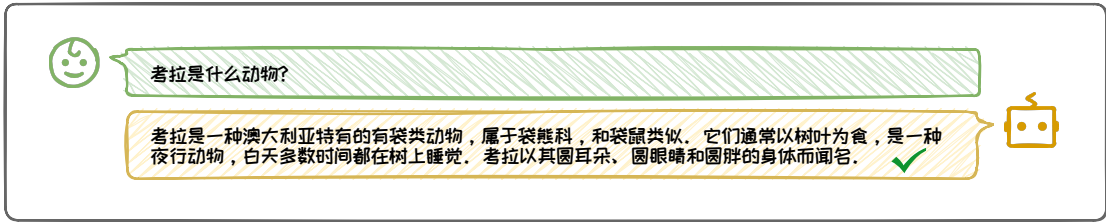


图 6.20: 模型对流行知识的回答示例。

频率与模型对该知识的记忆程度是正相关的 [22]。所以，我们可以通过判断训练数据中是否包含相应的知识来判断模型是否掌握了相应的知识。然而，这种方式存在一定的局限性。首先，由于大语言模型的训练数据规模已经达到了数万亿级别，因此这种统计的手段非常耗时。此外，对于许多商业大语言模型，例如 ChatGPT、GPT4 等，它们的训练数据并不是公开可获取的，在这种情况下，此方案无法执行。

(3) 构造伪训练数据统计量

当训练数据不可获取时，我们可以设计伪训练数统计量来拟合训练数据的相关情况。比如，由于模型对训练数据中低频出现的知识掌握不足，而对更“流行”（高频）的知识掌握更好，因此实体的流行度可以作为伪训练数据统计量。代表性工作 [33] 利用 **Wikipedia** 的页面浏览量来衡量实体的流行度，浏览量越大，表明该实体越流行，也就更可能被大语言模型所记忆。图 6.20 和图 6.21 分别展示了流行与不流行的知识的例子。从这两个例子中可以发现，对于流行知识**考拉**，模型能够直接给出正确的回答，而对于不流行知识**考拉的基因数量**，模型给出了错误的答案（正确答案应为 26,558）。由此可见，实体的流行度确实一定程度上反映了模型的内部知识，因此，可以通过设定一个流行度阈值来判别模型是否具备相应的内部知识。然而，这种流行度的定义依赖于数据形成时间，并且未必能精准拟合训练数据的分布，因此存在一定的局限性。

2. 内部观测法

为了进一步深入了解大语言模型的内部知识，在模型参数可访问的情况下，可

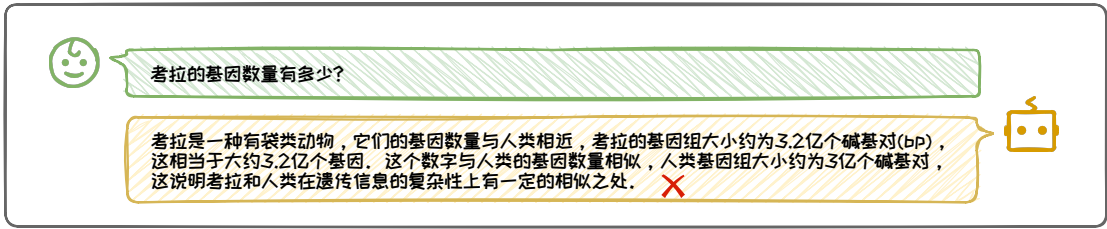


图 6.21: 模型对不流行知识的回答示例。

以通过观测模型内部的隐藏状态来更精确地评估其知识掌握情况。其可以类比人类测谎的过程，科学家通过分析脑电波、脉搏、血压等人类内部状态变化来推断大脑的活动和认知状态，从而判断其是否在说谎或隐藏某些信息。同样地，对于大语言模型，可通过分析模型在生成时每一层的隐藏状态变化，比如注意力模块的输出、多层感知器 (MLP) 层的输出与激活值变化等，来进行评估其内部知识水平。这是因为大语言模型在生成文本时，是对输入序列进行建模和预测，模型内部状态的变化反映了模型对当前上下文理解和下一步预测的确定性。如果模型表现出**较高的内部不确定性**，如注意力分布较为分散、激活值变化较大等，就可能对当前上下文缺乏充分的理解，从而无法做出有把握的预测。

由于**模型的内部知识检索主要发生在中间层的前馈网络中** [36]，因此在处理包含或不包含内部知识的不同问题时，模型的中间层会展现出不同的动态变化。基于这一特性，我们可以训练分类器进行判别，这种方法被称为探针。例如，Liang 等人 [30] 针对三种类型的内部隐藏状态设计了探针实验，分别是注意力层输出 (Attention Output)、MLP 层输出 (MLP Output) 和隐层状态 (Hidden States)，如图 6.22 所示。对于每个输入问题，研究者利用训练好的探针，即线性分类器，来根据问题所对应的内部表示预测该问题是属于模型“已知”（即模型具备相关知识）还是“未知”（即模型缺乏相关知识）。结果显示，不同大语言模型在利用中间层的内部表示进行分类时，均能够实现较高的分类准确率。这表明中间层的内部隐藏状态能够有效地反映模型对问题的理解和相关知识储备。

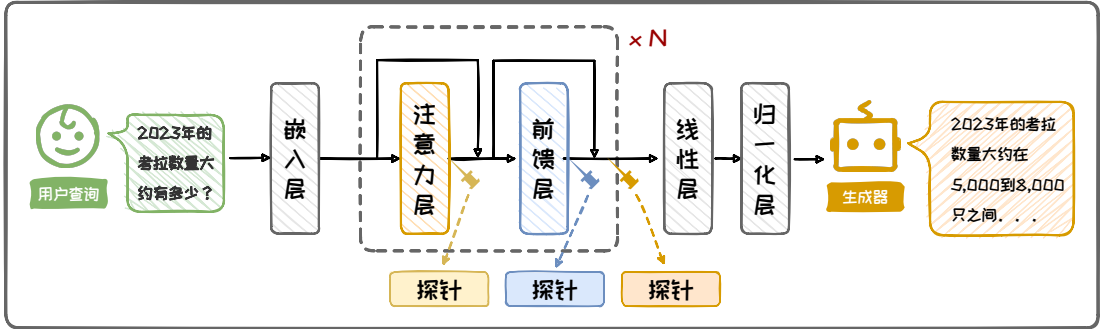


图 6.22: 模型内部状态探针。

然而，这种依赖于内部状态的检测方法也存在一定的局限性，它并不适用于黑盒语言模型，因为我们无法直接访问其内部隐藏状态。另外，模型的输入也需要仔细设计，因为模型有时所展现出的不确定性，可能并非源于对问题的知识缺失，而是问题本身固有的模糊性或歧义性所致。总体而言，基于内部状态评估内部知识的工作目前尚处于初步探索阶段，具体的方案设计有待进一步完善，例如如何构建模型“已知”和“未知”问题的数据集、如何量化内部状态的不确定性、不同内部表示的比对方法，如何设计内部状态检测方案等。我们需要在更广泛的数据集和更多样的模型架构上展开研究，以验证这一方法的有效性和普适性。但是，这是一条充满潜力的新路径，有望为我们从内部视角深入了解大语言模型的知识提供新的视角与方法。

6.4.2 何处增强

在确定大语言模型需要外部知识后，我们需要考虑在何处利用检索到的外部知识，即**何处增强**的问题。得益于大语言模型的上下文学习能力、注意力机制的可扩展性以及自回归生成能力，其输入端、中间层和输出端都可以进行知识融合操作。在输入端，可以将问题和检索到的外部知识拼接在 **Prompt** 中，然后输入给大语言模型；在中间层，可以采用**交叉注意力**将外部知识直接编码到模型的隐藏状

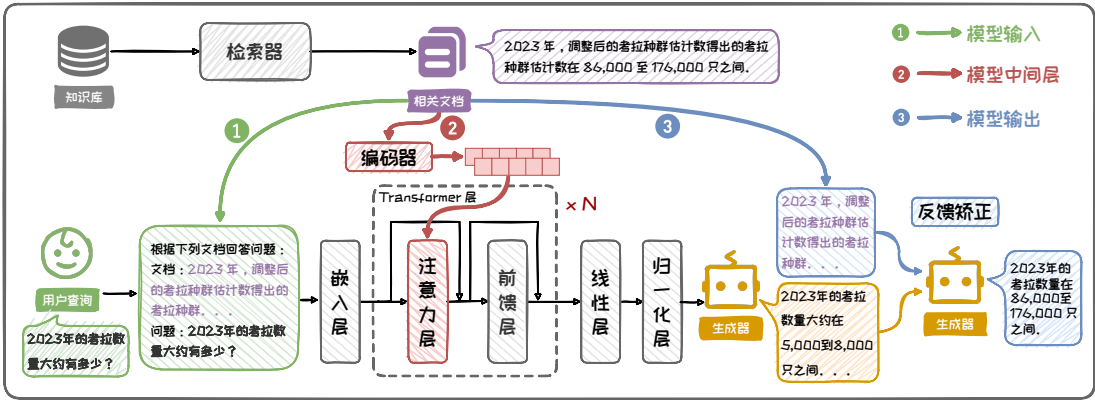


图 6.23: 增强实施位置示意图。

态中；在输出端，可以利用外部知识对生成的文本进行后矫正。图 6.23 通过一个例子展示了上述三种增强位置的实施方案。三种增强位置分别具有不同的优缺点，适合不同的场景。本小节将对其进行逐一介绍。

(1) 在输入端增强

在输入端增强的方法直接将检索到的外部知识文本与用户查询拼接到 Prompt 中，然后输入给大语言模型。其是当前主流的增强方法。此方式的重点在于 Prompt 设计以及检索到的外部知识的排序。良好的 Prompt 设计和外部知识排序，可以使模型更好地理解、利用外部知识。在设计 Prompt 的过程中，可以运用 CoT [54] 等 Prompt 技巧，具体方法可参阅本书第三章的 Prompt 工程内容。

在输入端增强的方法直观且易于实现。模型可以直接从输入的上下文中提取到所需信息，无需复杂的处理或转换。然而，当检索到的文本过长时，可能导致输入序列过长，甚至超出模型的最大序列长度限制。这给模型的上下文理解带来挑战，并且还会增加模型推理计算成本、增加其计算负担。这种方法对大语言模型的长文本处理能力和上下文理解能力要求较高。

(2) 在中间层增强

在中间层增强增强的方法利用注意力机制的灵活性，先将检索到的外部知识转换为向量表示，然后将这些向量插入通过交叉注意力融合到模型的隐藏状态中。

在第6.2节中介绍的 Retro [6] 方法，就是采用这种方式的典型代表。这种方法能够更深入地影响模型的内部表示，可能有助于模型更好地理解 and 利用外部知识。同时，由于向量表示通常比原始文本更为紧凑，这种方法可以减少对模型输入长度的依赖。然而，这种方法需要对模型的结构进行复杂的设计和调整，无法应用于黑盒模型。

(3) 在输出端增强

在输出端增强的方法利用检索到的外部知识对大语言模型生成的文本进行校准，是一种后处理的方法。在此类方法中，模型首先在无外部知识的情况下生成一个初步回答，然后再利用检索到的外部知识来验证或校准这一答案。校验过程基于生成文本与检索文本的知识一致性对输出进行矫正。矫正可以通过将初步回答与检索到的信息提供给大模型，让大模型检查并调整生成的回答来完成。例如，Yu 等人提出的 REFEED 框架 [59] 是此类方法典型代表。这种方法的优点是可以确保生成的文本与外部知识保持一致，提高答案的准确性和可靠性。然而，其效果在很大程度上依赖于检索到的外部知识的质量和相关性。若检索到的文档不准确或不相关，则会导致错误的校准结果。

综上，上述三种增强方式各有优劣，在实际应用中，我们可以根据具体的场景和需求，灵活选择不同的方案。并且，由于上述三种方案是相互独立的，它们也可以组合使用，以实现更优的增强效果。

6.4.3 多次增强

在实际应用中，用户对大语言模型的提问可能是复杂或模糊的。复杂问题往往涉及多个知识点，需要多跳（multi-hop）的理解；而模糊问题往往指代范围不明，难以一次就理解问题的含义。对于复杂问题和模糊问题，我们难以通过一次检索增强就确保生成正确，多次迭代检索增强在所难免。处理复杂问题时，常采用分解式

增强的方案。该方案将复杂问题分解为多个子问题，子问题间进行迭代检索增强，最终得到正确答案。处理模糊问题时，常采用**渐进式增强**的方案。该方案将问题的不断细化，然后分别对细化的问题进行检索增强，力求给出全面的答案，以覆盖用户需要的答案。本小节将对这两种方案展开介绍。

1. 分解式增强

复杂问题通常包含多个知识点。例如，在“世界上睡眠时间最长的动物爱吃什么？”这个问题中，包含着“世界上睡眠时间最长的动物是什么？”和“这个动物爱吃什么？”两个知识点。对复杂问题进行作答，需要多跳理解。因此，在复杂问题的检索增强中，我们通常无法仅通过一次检索增强就得到满意的答案。在这种情况下，模型可以将多跳问题分解为一个个子问题，然后在子问题间迭代地进行检索增强，最后得出正确结论，即分解式增强。

DEMONSTRATE-SEARCH-PREDICT (DSP) [25] 是一种具有代表性的分解式增强框架。该框架主要包含以下三个模块：（1）**DEMONSTRATE** 模块，通过上下文学习的方法，将复杂问题分解为子问题；（2）**SEARCH** 模块，对子问题进行迭代检索增强，为最终决策提供综合的外部知识；（3）**PREDICT** 模块，根据 **SEARCH** 提供的综合外部知识生成最终回答。

如图 6.24 所示，以“世界上睡眠时间最长的动物爱吃什么？”为例对 DSP 的流程进行介绍。首先，**DEMONSTRATE** 模块会在训练集中寻找类似问题的示例，以此来演示问题如何被分解。这些示例将指导大语言模型生成针对子问题的精确查询。例如，第一个子问题是：“世界上睡眠时间最长的动物是什么？”。随后，**SEARCH** 模块将利用第一个子问题来检索相关信息，从而确定睡眠最长的动物是考拉，并基于这一新信息形成第二个子问题：“考拉爱吃什么？”，随后再次执行检索，以找到描述考拉饮食习惯的相关文本。最终，在 **PREDICT** 模块中，大语言模型将综合所有信息得出最终答案，即“世界上睡眠时间最长的动物是考拉，爱吃桉树叶。”

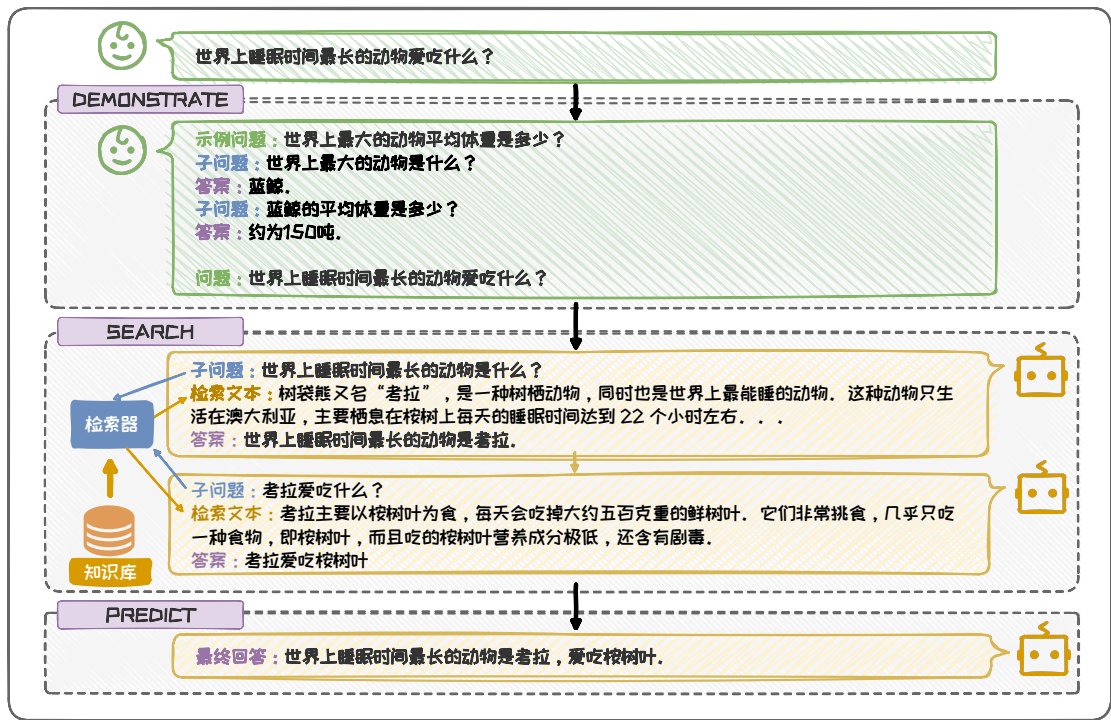


图 6.24: DSP 流程示意图。

分解式增强将复杂问题化整为零，降低了单次检索增强的难度。其性能很大程度上取决于子问题分解的质量。不同领域的复杂问题可能有着不同的分解范式，因此常常需要根据具体任务对问题分解方案进行设计。

2. 渐进式增强

在模糊问题中，问题主体通常指代不明，容易引发歧义。例如，在“国宝动物爱吃什么？”这个问题中，由于不同国家国宝动物不同，且部分国家的国宝动物不止一种，因此我们无法确定询问的是其中的哪一种动物。在处理这样的模糊问题时，我们可以对问题进行渐进式地拆解、细化，然后对细化后的问题进行检索，利用检索到的信息增强大模型。这种渐进式的增强方法可以帮助大语言模型掌握更全面的信息。

TREE OF CLARIFICATIONS (TOC) [26] 是渐进式增强的代表性框架。该框架通过递归式检索来引导大语言模型在**树状结构**中探索给定模糊问题的多种澄

清路径。图 6.25 展示了 TOC 框架的整个工作流，我们以“国宝动物爱吃什么？”这一问题为例进行说明。TOC 框架首先启动第一轮检索，根据检索到的相关文档和原始问题，生成一系列具体的细化问题，例如：“中国的国宝动物爱吃什么？”，“澳洲的国宝动物爱吃什么？”。在此过程中，框架会根据细化问题与原问题的相关性 & 知识一致性进行剪枝。随后，针对每个细化问题，框架独立展开深入的检索并对问题进一步细化，例如：“澳洲的国宝动物考拉爱吃什么？”。最终，我们能够构建出多条完整的知识路径，每条路径的末端（叶节点）都代表了对原始问题的不同但是有效的解答。通过整合所有有效的叶节点，我们能够得出一个精确且全面的长回答，例如对此问题我们得到：“中国的国宝动物熊猫爱吃竹子；澳洲的国宝动物考拉爱吃桉树叶；澳洲的国宝动物袋鼠爱吃杂草和灌木……”

这种渐进式的检索方法能够显著改善大语言模型对模糊问题的生成效果。然而，其需要进行多轮检索并生成多个答案路径，整个系统的计算量会随着问题复杂度呈指数级增长。此外，多轮的检索与生成不仅会带来显著的时间延迟，多个推理路径还为推理带来不稳定性，容易引发错误。

6.4.4 降本增效

检索出的外部知识通常包含大量原始文本。将其通过 Prompt 输入给大语言模型时，会大幅度增加输入 Token 的数量，从而增加了大语言模型的推理计算成本。此问题可从去除冗余文本与复用计算结果两个角度进行解决。本小节将对这两个角度分别展开介绍。

1. 去除冗余文本

在 RAG 中，检索出的原始文本通常包含大量的无益于增强生成的冗余信息。这些冗余信息不仅增加了输入 Token 的长度，而且还有可能对大模型产生干扰，导致生成错误答案。去除冗余文本的方法通过对检索出的原始文本的词句进行过

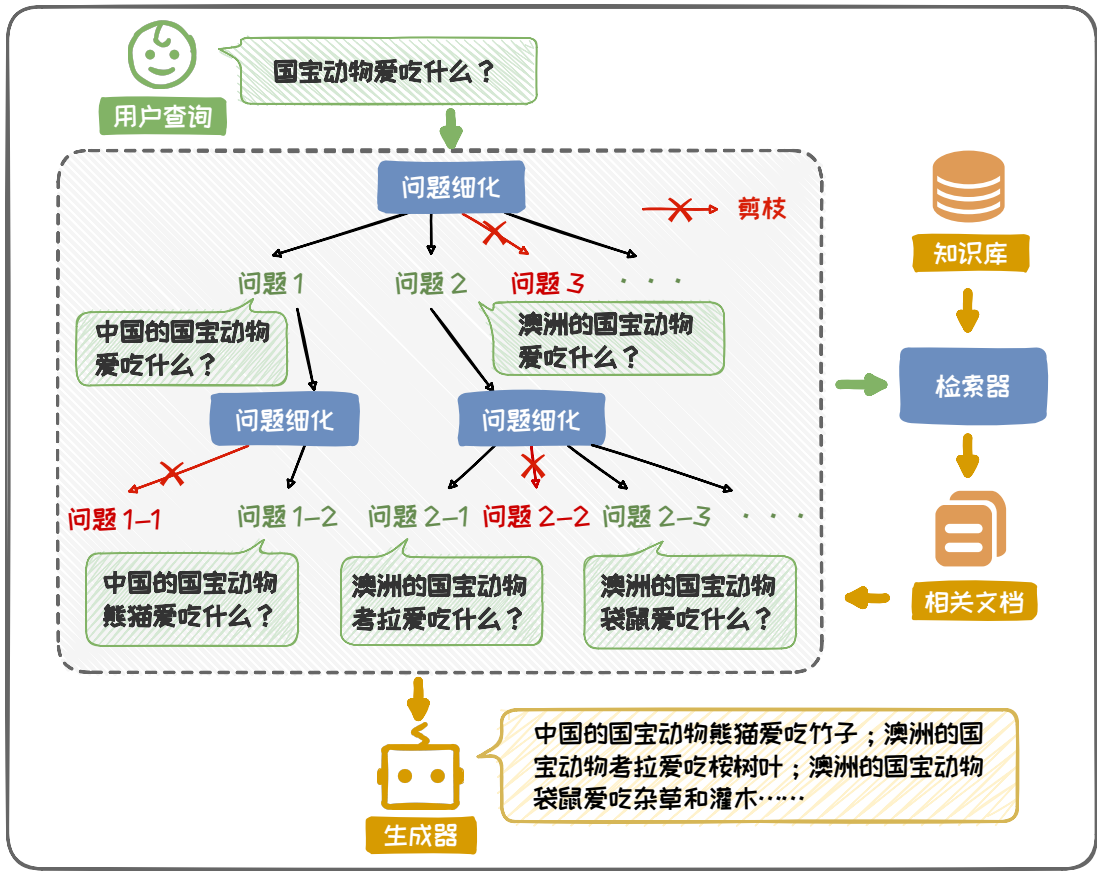


图 6.25: TOC 框架流程示意图。

滤，从中选择出部分有益于增强生成的部分。去除冗余文本的方法主要分为三类：Token 级别的方法，子文本级别的方法以及全文本级别的方法。这些方法通过不同的机制来筛选和优化检索出的原始文本，以减少无益信息，确保生成内容的相关性和准确性。以下分别对这三类方法分别展开介绍。

Token 级别的方法通过对 Token 进行评估，对文本中不必要的 Token 进行剔除。困惑度是判断 Token 重要性的重要指标。直观上说，如果一个 Token 的困惑度低，这意味着模型有很高的概率预测到这个 Token，表明该 Token 易于预测且较为普遍，因此它携带的新信息量较少，可能是冗余的；反之，如果一个 Token 的困惑度高，则表明这个 Token 携带了更多的信息量。LongLLMLingua [20] 框架利用小模型评

估 Token 的困惑度，然后基于困惑度删除冗余文本。其首先进行问题感知的**粗粒度压缩**，即在给定问题条件下通过计算文档中所有 Token 困惑度的均值来评估文档的重要性，困惑度越高表示文档信息量越大。随后，执行问题感知的**细粒度压缩**，即进一步计算文档中每个 Token 的困惑度并去除其中低困惑度的 Token。此外，论文还引入了**文档重排序机制**、**动态压缩比率**以及**子序列恢复机制**，确保重要文档和文档中的重要信息被有效利用。

子文本级别的方法通过对子文本进行打分，对不必要的子文本成片删除。**FIT-RAG** [35] 是子文本级别方法中的代表性方法。该方法中采用了预先训练的双标签子文档打分器，从两个维度评估文档的有用性：一是**事实性**，即文档是否包含查询的答案信息；二是**模型的偏好程度**，即文档是否易于模型理解。对于检索到的文档，首先利用滑动窗口将其分割成多个子文档，然后使用双标签子文档打分器对这些子文档分别进行评分。最后，删除掉评分较低子文档，从而有效地去除冗余文本。

全文本级别的方法直接从整个文档中抽取出重要信息，以去除掉冗余信息。经典方法 **PRCA** [57] 通过训练能够提炼重点内容的信息提取器对文本中的重要信息进行提取。该方法分为两个阶段，上下文提取阶段与奖励驱动阶段。在**上下文提取阶段**，以最小化压缩文本与原输入文档之间的差异为目标，对信息提取器进行监督学习训练，学习如何将输入文档精炼为信息丰富的压缩文本。在**奖励驱动阶段**，大语言模型作为奖励模型，其根据压缩文本生成的答案与真实答案之间的相似度作为奖励信号，通过强化学习对信息提取器进行优化。最终得到的信息提取器可以直接将输入文档转化为压缩文本，端到端地去除冗余文本。

2. 复用计算结果

除了对冗余信息进行筛除，我们还可以对计算必需的中间结果进行复用，以优化 RAG 效率。在大语言模型进行推理的自回归过程中，每个 Token 都需要用到

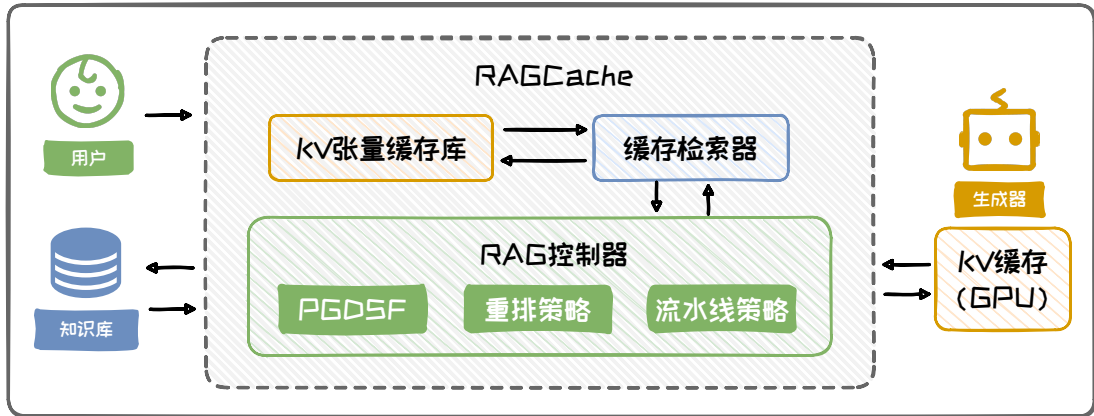


图 6.26: RAGCache 框架流程示意图。

之前 Token 注意力模块涉及的 Key 和 Value 的结果。为了避免对每个 Token 都重新计算前面的 Key 和 Value 的结果，我们可以将之前计算的 Key 和 Value 的结果进行缓存（即 KV-cache），在需要是直接从 KV-cache 中调用相关结果，从而避免重复计算。但是，随着输入文本长度的增加，KV-cache 的 GPU 显存占用会随之剧增，甚至会远远超过模型参数所占用的显存大小 [46]。然而，RAG 的输入通常包含检索出来的文本，导致输入文本很长，导致 RAG 中的 KV-cache 存储成本高昂。

不过，在 RAG 中，不同用户查询经常检索到相同的文本，而且常见的查询通常数量有限。因此，我们可以将常用的重复文本的 KV-cache 进行复用，避免每次查询都对其进行计算，以降低存储成本。基于此，RAGCache [21] 设计了一种 RAG 系统专用的多级动态缓存机制，如图 6.26 所示。RAGCache 系统由三个核心部分组成：KV 张量缓存库、缓存检索器与 RAG 控制器。其中，KV 张量缓存库采用树结构来缓存所计算出的文档 KV 张量，其中每个树节点代表一个文档；缓存检索器则负责在缓存库中快速查找是否存在所需的缓存节点；而 RAG 控制器作为系统的策略中枢，负责制定核心的缓存策略。具体而言，RAG 控制器首先采用了一种前缀感知的贪婪双重尺度频率（PGDSF）替换策略。该策略综合考虑了文档节点的访问频率、大小、访问成本以及最近访问时间，使得频繁使用的文档能够被快速检索